Entrega Lab CI/CD - Desafio RH

1. Link do repositório do projeto

https://github.com/anablefari/desafio-rh

2. Arquivo da pipeline

.github/workflows/lab-cicd.yml

O conteúdo integral do workflow é apresentado nas páginas seguintes.

2. Arquivo da pipeline - parte 1 de 5

```
001  name: Lab CI/CD - Desafio RH
002
003  on:
004    push:
005      branches: [main]
006    pull_request:
007      branches: [main]
008
009  permissions:
010    contents: read
011
012  concurrency:
013    group: lab-cicd-${{ github.ref }}
014    cancel-in-progress: true
015
016  jobs:
017    ci-backend:
018      name: Backend - testes e imagem Docker
019      runs-on: ubuntu-latest
020      timeout-minutes: 15
021      defaults:
022        run:
023          working-directory: backend
024
025      steps:
026        - name: Checkout do repositório
027          uses: actions/checkout@v7
028
029        - name: Configurar Java 21
030          uses: actions/setup-java@v6
031          with:
032            distribution: temurin
033            java-version: '21'
034            cache: maven
035            cache-dependency-path: backend/pom.xml
036
037        - name: Compilar e executar testes
038          run: bash ./mvnw --batch-mode --no-transfer-progress clean verify
```

2. Arquivo da pipeline - parte 2 de 5

```
039
040       - name: Configurar Docker Buildx
041         uses: docker/setup-buildx-action@v4
042
043       - name: Validar build da imagem Docker
044         uses: docker/build-push-action@v7
045         with:
046           context: backend
047           file: backend/Dockerfile
048           platforms: linux/amd64
049           push: false
050           cache-from: type=gha,scope=backend
051           cache-to: type=gha,mode=max,scope=backend
052
053   ci-frontend:
054     name: Frontend - lint e build
055     runs-on: ubuntu-latest
056     timeout-minutes: 10
057     defaults:
058       run:
059         working-directory: frontend
060
061     steps:
062       - name: Checkout do repositório
063         uses: actions/checkout@v7
064
065       - name: Configurar Node.js 24
066         uses: actions/setup-node@v7
067         with:
068           node-version: '24'
069           cache: npm
070           cache-dependency-path: frontend/package-lock.json
071
072       - name: Instalar dependências
073         run: npm ci
074
075       - name: Analisar código com ESLint
076         run: npm run lint
```

2. Arquivo da pipeline - parte 3 de 5

```
077
078       - name: Compilar frontend
079         run: npm run build
080
081   cd-backend:
082     name: CD Backend - Docker Hub e Render
083     needs: [ci-backend, ci-frontend]
084     if: github.event_name == 'push' && github.ref == 'refs/heads/main'
085     runs-on: ubuntu-latest
086     timeout-minutes: 15
087     environment: production-backend
088     env:
089       DOCKER_IMAGE: ${{ vars.DOCKERHUB_IMAGE }}
090       DOCKERHUB_USERNAME: ${{ vars.DOCKERHUB_USERNAME }}
091       DOCKERHUB_TOKEN: ${{ secrets.DOCKERHUB_TOKEN }}
092       RENDER_DEPLOY_HOOK_URL: ${{ secrets.RENDER_DEPLOY_HOOK_URL }}
093
094     steps:
095       - name: Checkout do repositório
096         uses: actions/checkout@v7
097
098       - name: Validar configuração do backend
099         shell: bash
100         run: |
101           if [ -z "$DOCKER_IMAGE" ] || [ -z "$DOCKERHUB_USERNAME" ] || [ -z "$DOCKERHUB_TOKEN" ] || [ -z "$RENDE
    R_DEPLOY_HOOK_URL" ]; then
102             echo "Configure DOCKERHUB_IMAGE e DOCKERHUB_USERNAME como Variables; DOCKERHUB_TOKEN e RENDER_DEPLOY
    _HOOK_URL como Secrets."
103             exit 1
104           fi
105
106       - name: Autenticar no Docker Hub
107         uses: docker/login-action@v4
108         with:
109           username: ${{ env.DOCKERHUB_USERNAME }}
110           password: ${{ env.DOCKERHUB_TOKEN }}
111
112       - name: Configurar Docker Buildx
```

2. Arquivo da pipeline - parte 4 de 5

```
        uses: docker/setup-buildx-action@v4

      - name: Publicar imagem do backend
        uses: docker/build-push-action@v7
        with:
          context: backend
          file: backend/Dockerfile
          platforms: linux/amd64
          push: true
          tags: |
            ${{ env.DOCKER_IMAGE }}:latest
            ${{ env.DOCKER_IMAGE }}:${{ github.sha }}
          cache-from: type=gha,scope=backend
          cache-to: type=gha,mode=max,scope=backend

      - name: Publicar a nova imagem no Render
        shell: bash
        run: |
          curl --fail --silent --show-error \
            --request POST \
            --get \
            --data-urlencode "imgURL=docker.io/${DOCKER_IMAGE}:${GITHUB_SHA}" \
            "$RENDER_DEPLOY_HOOK_URL"

  cd-frontend:
    name: CD Frontend - Vercel
    needs: [ci-backend, ci-frontend]
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    timeout-minutes: 15
    environment: production-frontend
    env:
      VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
      VERCEL_ORG_ID: ${{ vars.VERCEL_ORG_ID }}
      VERCEL_PROJECT_ID: ${{ vars.VERCEL_PROJECT_ID }}
      VITE_API_URL: ${{ vars.BACKEND_PUBLIC_URL }}

    steps:
```

2. Arquivo da pipeline - parte 5 de 5

```
151       - name: Checkout do repositório
152         uses: actions/checkout@v7
153
154       - name: Configurar Node.js 24
155         uses: actions/setup-node@v7
156         with:
157           node-version: '24'
158
159       - name: Validar configuração do frontend
160         shell: bash
161         run: |
162           if [ -z "$VERCEL_TOKEN" ] || [ -z "$VERCEL_ORG_ID" ] || [ -z "$VERCEL_PROJECT_ID" ] || [ -z "$VITE_API
    _URL" ]; then
163             echo "Configure VERCEL_TOKEN como Secret; VERCEL_ORG_ID, VERCEL_PROJECT_ID e BACKEND_PUBLIC_URL como
     Variables."
164             exit 1
165           fi
166
167       - name: Instalar Vercel CLI
168         run: npm install --global vercel@latest
169
170       # O projeto Vercel usa Root Directory = frontend. A CLI parte da raiz
171       # para evitar que a configuração resulte no caminho frontend/frontend.
172       - name: Carregar configurações de produção
173         run: vercel pull --yes --environment=production --token="$VERCEL_TOKEN"
174
175       - name: Preparar build de produção
176         run: vercel build --prod --token="$VERCEL_TOKEN"
177
178       - name: Publicar frontend
179         run: vercel deploy --prebuilt --prod --token="$VERCEL_TOKEN"
```

3. Prompts utilizados

Prompt 1

Use como referência os PDFs 07 - CI - Integração Contínua, 08 - CI - GitAction_Docker e Lab CICD-Pipeline. Analise este projeto full-stack, com frontend React/TypeScript e backend Spring Boot/Maven, e implemente do zero o laboratório de CI/CD. Crie um único workflow do GitHub Actions com CI independente para frontend e backend, validação da imagem Docker do backend e deployment automático somente após a aprovação das duas CIs. Preserve a estrutura existente do projeto e documente as configurações necessárias.

Prompt 3

Configure a entrega contínua do backend e do frontend no workflow. No backend, publique no Docker Hub uma imagem linux/amd64 com as tags latest e SHA do commit e, em seguida, acione o deploy automático no Render. No frontend, gere e publique o build de produção na Vercel apontando para a URL pública do backend. Use GitHub Secrets para tokens e URLs sensíveis, Variables para identificadores e endereços públicos, environments separados e condições que impeçam deployment em pull requests.

Prompt 4

Valide a pipeline completa e corrija os problemas encontrados durante a execução. Verifique sintaxe YAML, versões das actions, cache, permissões mínimas, timeouts, dependências entre jobs e formato das variáveis. Diagnostique falhas de tag do Docker e de identificação do projeto Vercel, ajuste o CORS do backend para aceitar localhost e deployments HTTPS da Vercel, execute os testes e builds novamente e registre evidências reais do GitHub Actions, Docker Hub, Render, API e Vercel.

4. Registro das alterações realizadas

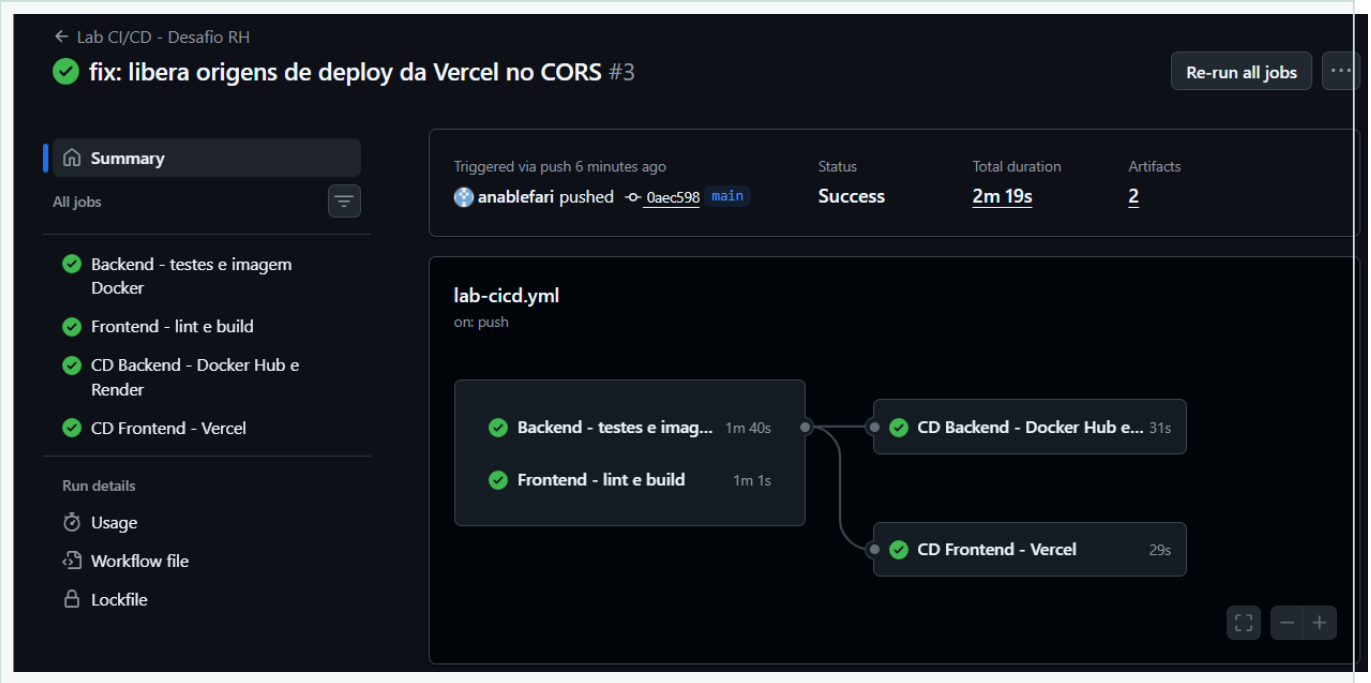
Alteração	Registro
Workflow	Substituição da pipeline anterior por .github/workflows/lab-cicd.yml.
CI backend	Java 21, cache Maven, clean verify e validação da imagem Docker linux/amd64.
CI frontend	Node.js 24, cache npm, npm ci, ESLint e build Vite.
Imagem Docker	Publicação no Docker Hub com as tags latest e SHA do commit.
CD backend	Acionamento do Render após a publicação da imagem e o sucesso das duas CIs.
CD frontend	Vercel pull, build de produção e deploy prebuilt após o sucesso das duas CIs.
Segurança	Permissão contents: read, Secrets, Variables, environments e validação de configurações.
Resiliência	Timeouts, cache e cancelamento de execuções antigas da mesma referência.
Integração	CORS ajustado para localhost e subdomínios HTTPS da Vercel.
Versionamento	Implementação e correção final publicadas na main; execução validada no commit 0aec598.

5. Breve análise do processo

A implementação separa integração contínua e entrega contínua. Em pushes e pull requests para main, frontend e backend são validados em paralelo; os deployments só são executados em push na main e dependem do sucesso das duas CIs. A imagem do backend recebe as tags latest e SHA, o que mantém uma referência direta entre artefato e commit. Secrets e Variables evitam credenciais no repositório. Durante a validação real, quebras de linha em variáveis causaram tags Docker inválidas e falha de identificação do projeto Vercel; a correção confirmou a importância de revisar o valor exato das configurações. Por fim, o CORS foi ajustado para permitir os deployments dinâmicos da Vercel. A execução final aprovou os quatro jobs e os testes funcionais de listar, cadastrar, editar e excluir funcionários.

6. Evidências da execução correta da pipeline CI/CD

As imagens a seguir registram a execução do workflow e o funcionamento dos artefatos publicados em produção.



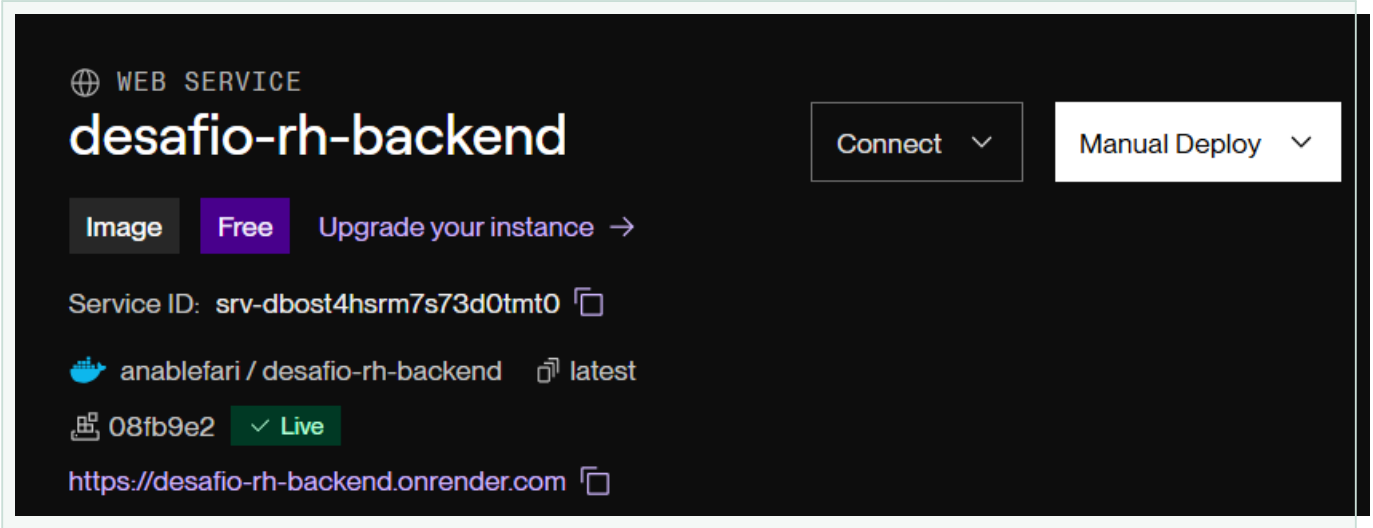
Evidência 1 - GitHub Actions: execução do commit 0aec598 concluída com status Success; CI de backend e frontend e CD para Render e Vercel aprovados.

6. Evidências da execução correta da pipeline CI/CD



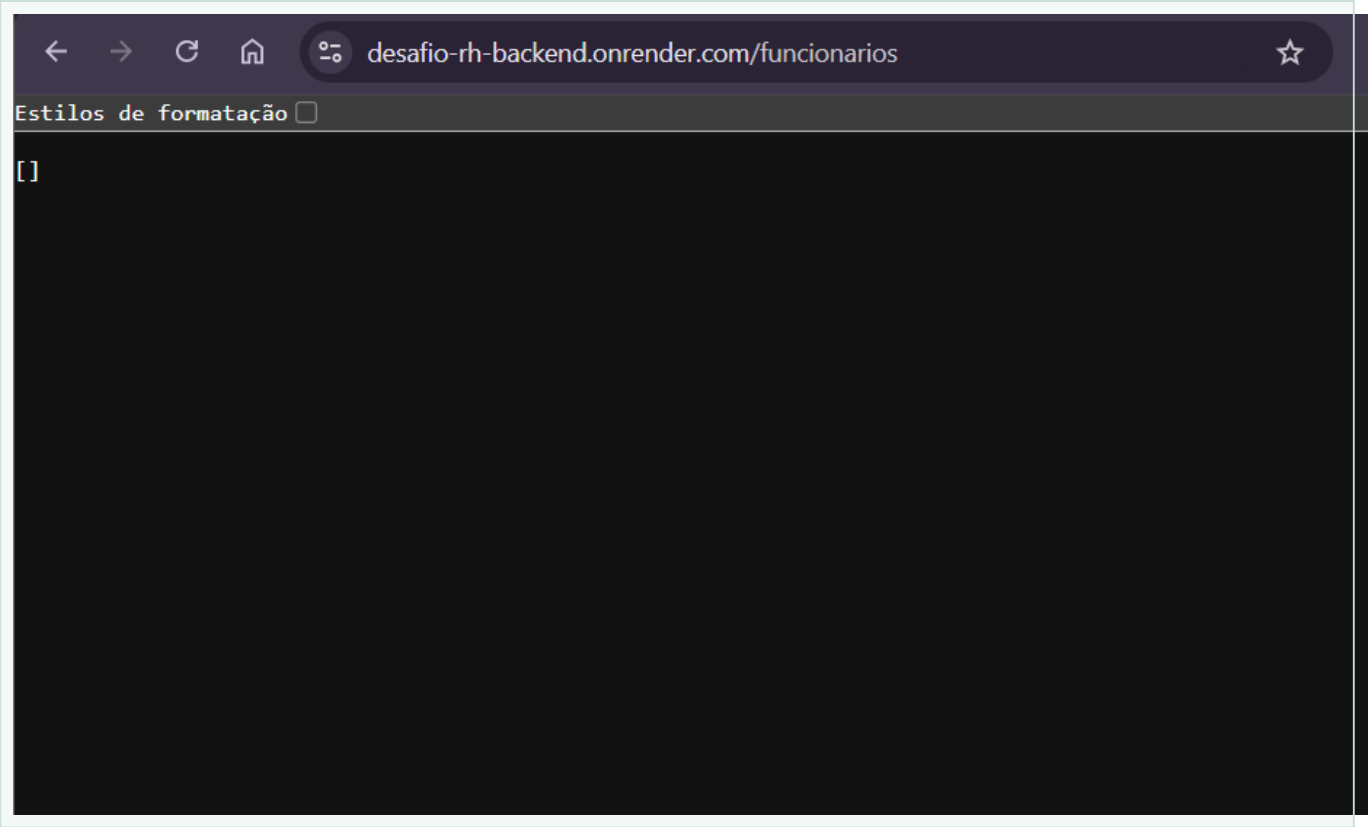
Evidência 2 - Docker Hub: imagem anablefari/desafio-rh-backend publicada com as tags imutável do commit e latest.

6. Evidências da execução correta da pipeline CI/CD



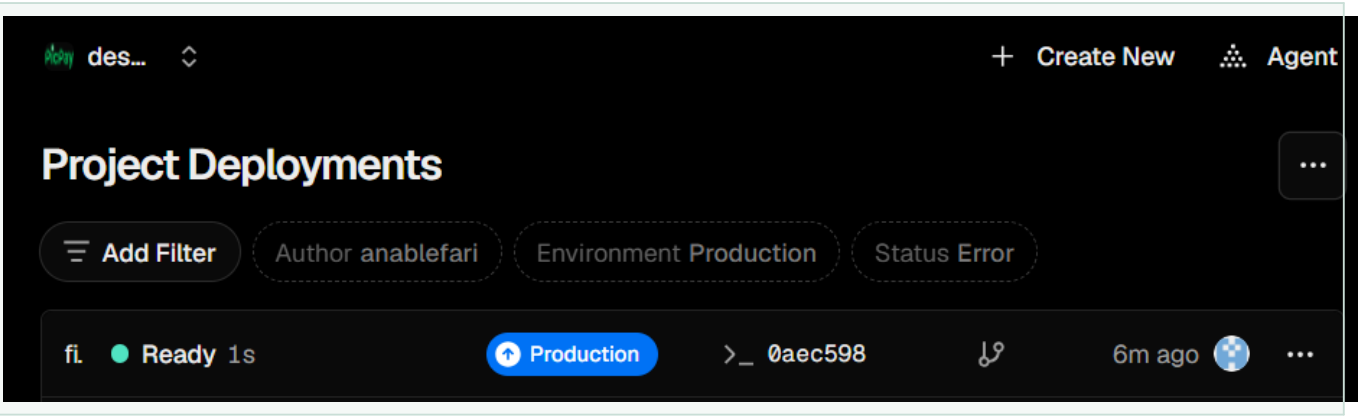
Evidência 3 - Render: serviço desafio-rh-backend executando a imagem do Docker Hub com estado Live.

6. Evidências da execução correta da pipeline CI/CD



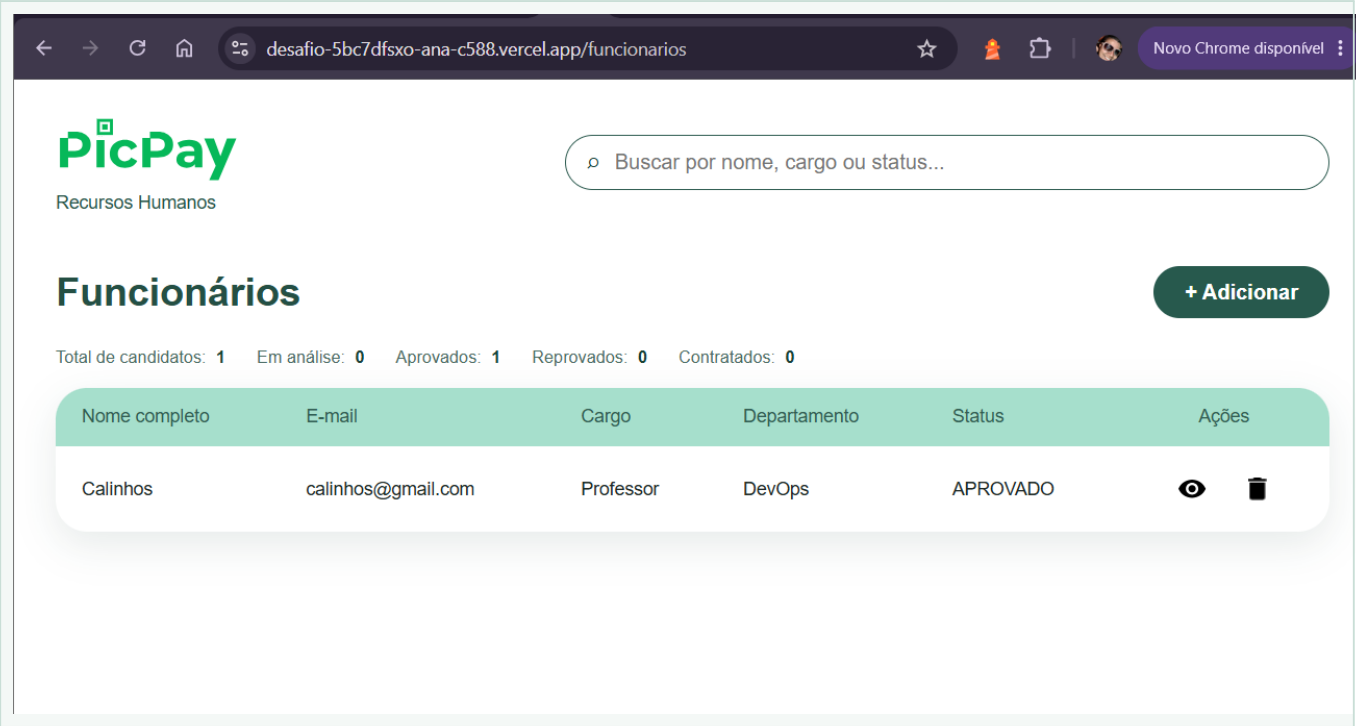
Evidência 4 - API em produção: endpoint público /funcionarios acessível no Render e retornando resposta JSON válida.

6. Evidências da execução correta da pipeline CI/CD



Evidência 5 - Vercel: deployment do frontend associado ao commit 0aec598 em Production com estado Ready.

6. Evidências da execução correta da pipeline CI/CD



Evidência 6 - Aplicação integrada: frontend publicado na Vercel consumindo o backend do Render e exibindo o funcionário cadastrado.